
Online Social Bookstore Application

Sprint Report

EVANGELOS BASDAVANOS 4962

KONSTANTINOS KAPSIMALIS 4698

ROMANOS KOTSIS 4714

VERSIONS HISTORY

Date	Version	Description	Author
11/4/24	V1.0	UC1-UC5	Evangelos Basdavanos
23/4/24	V2.0	UC6-UC9	Evangelos Basdavanos
16/5/24	V3.0	UC10-UC11	Evangelos Basdavanos

Introduction

This document provides information concerning the **3rd** sprint of the project.

1.1 Purpose

The purpose of the document is to explain the structure and the functionality of the project. This is done with the use cases, which describe the main functions of the program, and with the crc cards, which describe the responsibilities and collaborations of each class.

1.2 Document Structure

The rest of this document is structured as follows. Section 2 describes our Scrum team and specifies this Sprint's backlog. Section 3 specifies the main design concepts for this release of the project.

2 Scrum team and Sprint Backlog

2.1 Scrum team

Product Owner	Romanos Kotsis 4714
Scrum Master	Konstantinos Kapsimalis 4698
Development Team	Evangelos Basdavanos 4962 Konstantinos Kapsimalis 4698 Romanos Kotsis 4714

2.2 Sprints

Sprint No	Begin Date	End Date	Number of weeks	User stories
1	8/4/2024	11/4/2024	-	US1, US2, US3, US4
2	11/4/2024	23/4/2024	-	US5, US6, US7, US8, US9
3	24/4/2024	16/5/2024	-	US10, US11

3 Use Cases

3.1 SelectCreateNewAccount

Use case ID	UC1
Actors	User
Pre conditions	1. The user must have opened the app.
Main flow of events	1. The use case starts when the user clicks the “Create new account” button.
Post conditions	The system has opened the “Create New Account” page.

3.2 Login

Use case ID	UC2
Actors	User
Pre conditions	1. The user must have opened the app.
Main flow of events	1. The use case starts when the user enters his credentials. 2. The user clicks the “Login” button.
Alternative flow 1	1. The user hasn’t entered his credentials. 2. The user clicks the “Login” button. 3. An error message pops-up.
Alternative flow 2	1. The user has entered credentials, with which an account hasn’t been created. 2. The user clicks the “Login” button. 3. An error message pops-up.
Post conditions	The system has opened the “Home” page.

3.3 Logout

Use case ID	UC3
Actors	User
Pre conditions	1. The user must have logged in the app.
Main flow of events	1. The use case starts when the user clicks the “Logout” button.
Post conditions	The system has logged the user out and opens the “Login” page.

3.4 CreateNewAccount

Use case ID	UC4
Actors	User
Pre conditions	1. UC1 must have been performed.
Main flow of events	1. The use case starts when the user fills the fields with his information. 2. The user clicks the “Create Account” button.
Alternative Flow 1	1. The user hasn’t filled in all the mandatory fields. 2. The user clicks the “Create Account” button. 3. An error message pops up.
Alternative Flow 2	1. The user enters an existing username. 2. The user clicks the “Create Account” button. 3. An error message pops up.
Post conditions	The system has successfully created the new account and opens the “Login” page.

3.5 AddBookOffer

Use case ID	UC5
Actors	User
Pre conditions	1. UC2 must have been performed.
Main flow of events	1. The use case starts when the user clicks the “Make a Book Offer” button. 2. The system redirects the user to the make offer page. 3. The user fills in the details of the book in the fields. 4. The user clicks the “Add offer” button.
Alternative Flow 1	1. The user hasn’t filled in all the mandatory fields. 2. The user clicks the “Add offer” button. 3. An error message pops up.
Post conditions	The system has successfully added the new book offer in the database and opens the Homepage.

3.6 BrowseRequests

Use case ID	UC6
Actors	User
Pre conditions	1. UC2 must have been performed.
Main flow of events	1. The use case starts when the user is in the HomePage. 2. The systems displays the requests of other users, for this user’s book offers, automatically.
Post conditions	-

3.7 AcceptRequestAndNotify

Use case ID	UC7
Actors	User
Pre conditions	1. UC2 must have been performed. 2. There must have been at least one request for the user's book offers.
Main flow of events	1. The use case starts when the user clicks the "Accept" button on a request in the "My Requests" list. 2. The system deletes this specific book offer from the "My Book Offers" list and also deletes all the requests which has been made from this offer. 3. The system notifies the user that his request got accepted, with a message in his "My Notifications" list, with an acceptance message. It also notifies the rest of the users, who had made a request for this offer, with a rejection message.
Post conditions	The system has successfully updated the database.

3.8 GetInformationOfUser

Use case ID	UC8
Actors	User
Pre conditions	1. UC2 must have been performed.
Main flow of events	1. The use case starts when the user accepts a request. 2. The system displays a message in "My Notifications" list, with the information of the user that his request got accepted.
Post conditions	-

3.9 DeleteBookOffer

Use case ID	UC9
Actors	User
Pre conditions	1. UC2 must have been performed.

Main flow of events	1. The use case starts when the user clicks the “X” button next to a book offer in the “My Book Offers” list. 2. The system deletes the book offer from the “My Book Offers” list and also deletes all the requests which have been made for this offer in the “My Request” list.
Post conditions	The system has successfully updated the database.

3.10 SearchBookAndMakeRequest

Use case ID	UC10
Actors	User
Pre conditions	1. UC2 must have been performed.
Main flow of events	1. The use case starts when the user fills the search fields with either the book title or the book author or both of them. 2. The user clicks the “Search” button. 3. The system displays all the book offers which match the criteria in the “My Results” list, which are the offers which match exactly the criteria and the book offers which match them approximately. 4. The user clicks the “Make Request” button of a specific book offer in “My Results” list. 5. The system updates the “My Result” list.
Post conditions	The system has successfully updated the database.

3.11 BrowseBookOffersRecommendationsAndMakeRequest

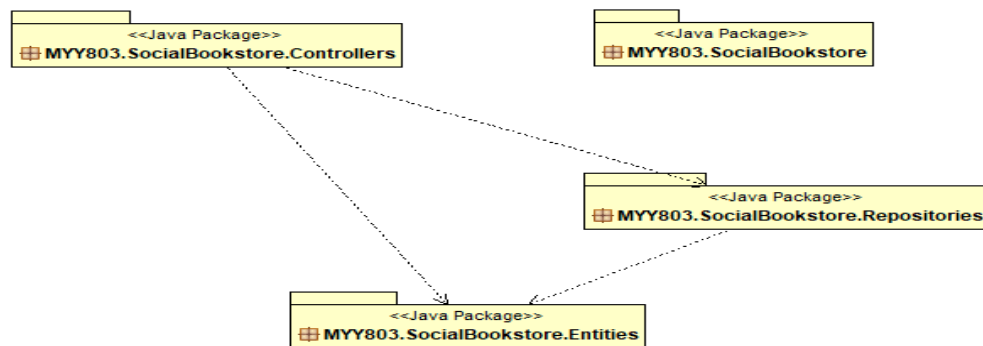
Use case ID	UC11
Actors	User
Pre conditions	1. UC2 must have been performed.
Main flow of events	1. The use case starts when the user clicks the “Search By My Favourite Categories” button. 2. The system displays all the book offers which have as a category at least

	one of the favourite categories of the user. 3. The user clicks the “Make Request” button of a specific book offer in “My Results” list. 4. The system updates the “My Result” list.
Alternative Flow 1	1. The use case starts when the user clicks the “Search By My Favourite Authors” button. 2. The system displays all the book offers which have as an author at least one of the favourite authors of the user. 3. The user clicks the “Make Request” button of a specific book offer in “My Results” list. 4. The system updates the “My Result” list.
Alternative Flow 2	1. The use case starts when the user clicks the “Search By My Favourite Categories & Authors” button. 2. The system displays all the book offers which have as an author at least one of the favourite authors of the user and as a category at least one of the favourite categories of the user. 3. The user clicks the “Make Request” button of a specific book offer in “My Results” list. 4. The system updates the “My Result” list.
Post conditions	The system has successfully updated the database.

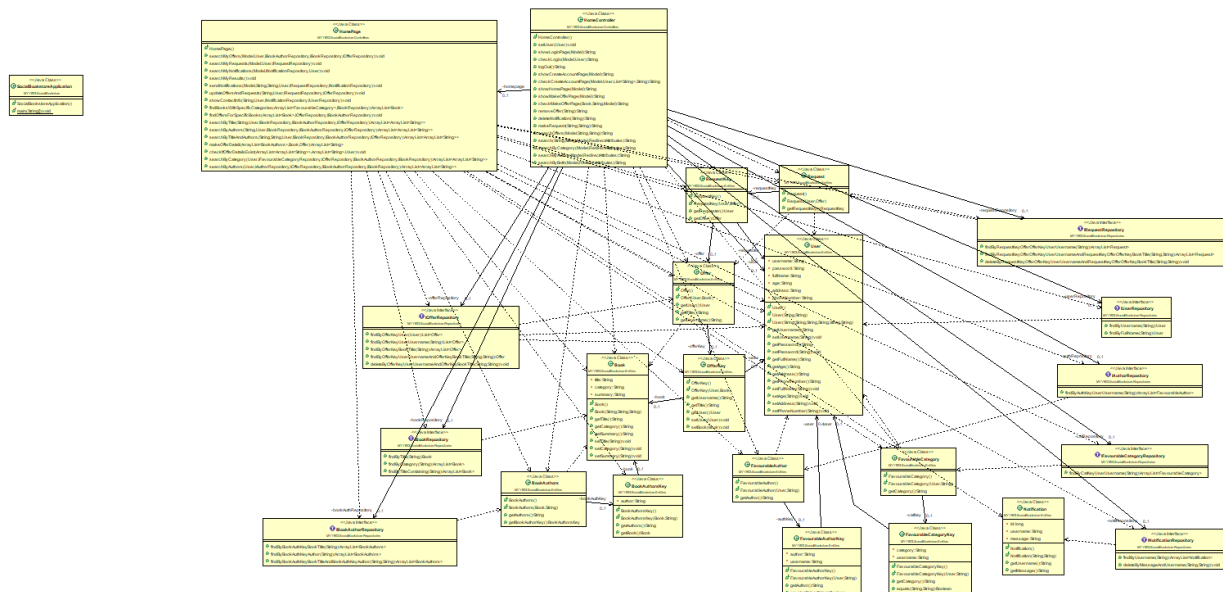
4 Design

4.1 Architecture

- Package Diagram

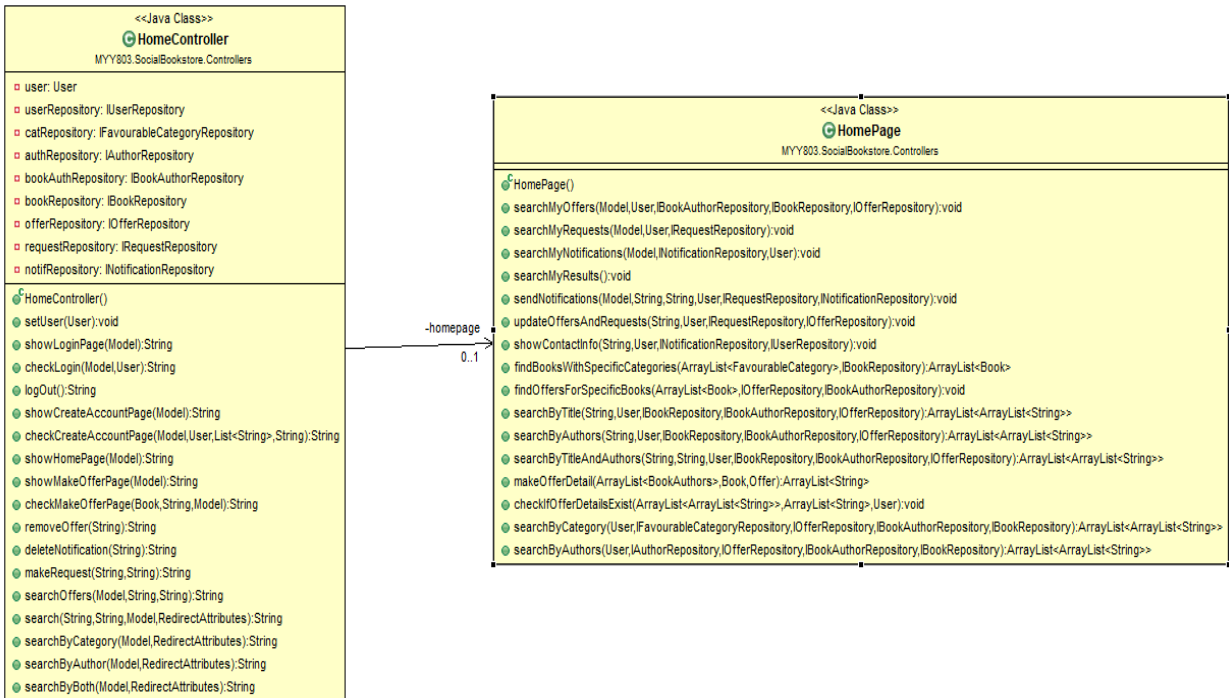


- Class Diagram

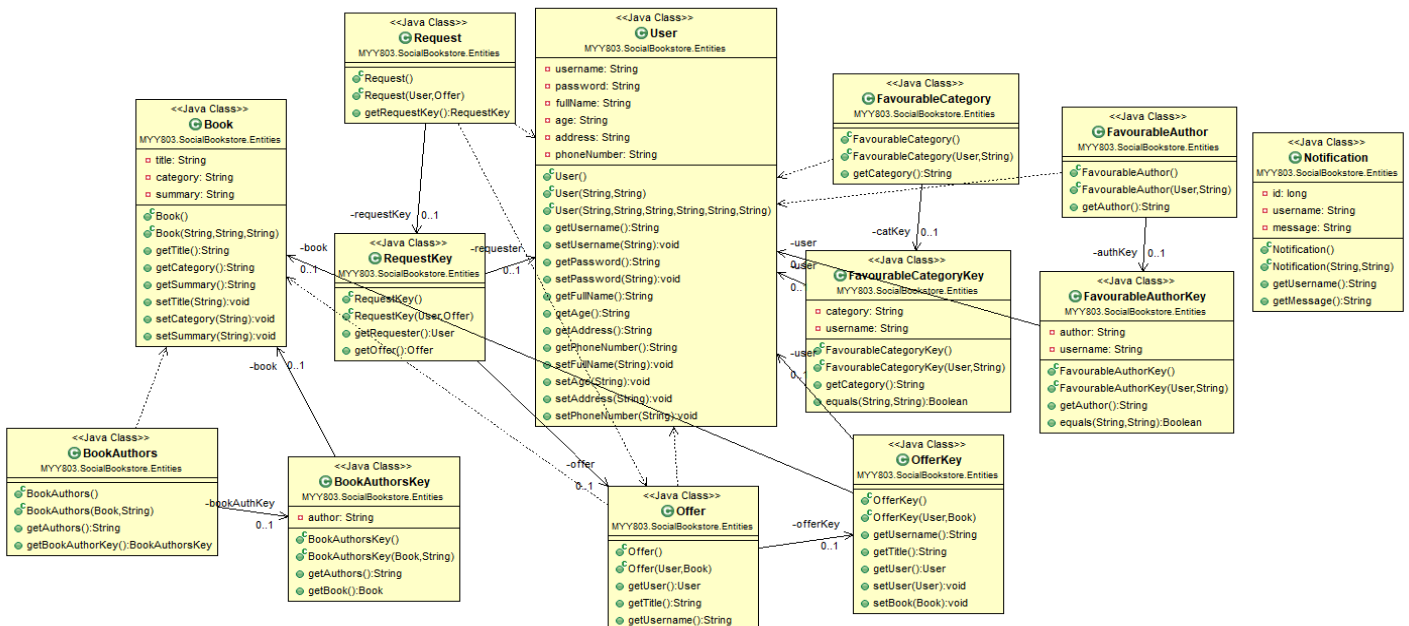


4.2 Design

■ Controllers Package Class Diagram



■ Entities Package Class Diagram



Class Name: HomeController	
Responsibilities: ▪ This class is responsible for the handling of the website. ▪ This class connects the backend with the frontend.	Collaborations: ▪ The class HomeController associates with the HomePage and User classes and with all the repository interfaces. ▪ The class HomeController depends on the rest of the classes except of the ...Key classes and the Notification class.

Class Name: HomePage	
Responsibilities: ▪ This class is responsible for implementing some of the methods that the homeController uses.	Collaborations: ▪ The class HomePage depends on all the repository interfaces and all the classes except of the ...Key classes.

Class Name: Book	
Responsibilities: ▪ This class is responsible for storing information about a book and adding it to the database.	Collaborations: ▪ The class Book doesn't depend or associate with any class.

Class Name: BookAuthors	
Responsibilities: ▪ This class is responsible for storing information about a book author and adding it to the database.	Collaborations: ▪ The class BookAuthors associates with the BookAuthorsKey class. ▪ The class BookAuthors depends on Book class.

Class Name: BookAuthorsKey	
Responsibilities: ▪ This class is responsible for creating a key for the BookAuthors entity.	Collaborations: ▪ The class BookAuthors associates with the Book class.

Class Name: FavourableAuthor	
Responsibilities: ▪ This class is responsible for storing information about the favourite authors of a user and adding it to the database.	Collaborations: ▪ The class FavourableAuthor associates with the FavourableAuthorKey class. ▪ The class FavourableAuthor depends on User class.

Class Name: FavourableAuthorKey	
Responsibilities: ▪ This class is responsible for creating a key for the FavourableAuthor entity.	Collaborations: ▪ The class FavourableAuthorKey associates with the User class.

Class Name: FavourableCategory	
Responsibilities: ▪ This class is responsible for storing information about the favourite categories of a user and adding it to the database.	Collaborations: ▪ The class FavourableCategory associates with the FavourableCategoryKey class. ▪ The class FavourableCategory depends on User class.

Class Name: FavourableCategoryKey	
Responsibilities: ▪ This class is responsible for creating a key for the FavourableCategory entity.	Collaborations: ▪ The class FavourableAuthorKey associates with the User class.

Class Name: Notification	
Responsibilities: ▪ This class is responsible for storing the messages of a user and adding it to the database.	Collaborations: ▪ The class Notification doesn't depend on or associate with any class.

Class Name: Offer	
Responsibilities: ▪ This class is responsible for storing the information of offers of a user about a book and adding it to the database.	Collaborations: ▪ The class Offer is associated with the OfferKey class. ▪ The class Offer depends on the User and Book classes.

Class Name: OfferKey	
Responsibilities: ▪ This class is responsible for creating a key for the Offer entity.	Collaborations: ▪ The class FavourableAuthorKey associates with the User and Book classes.

Class Name: Request	
Responsibilities: ▪ This class is responsible for storing the information of requests of users about a book and adding it to the database.	Collaborations: ▪ The class Request is associated with the RequestKey class. ▪ The class Request depends on the User and Offer classes.

Class Name: RequestKey	
Responsibilities: ▪ This class is responsible for creating a key for the Request entity.	Collaborations: ▪ The class RequestKey associates with the User and Offer classes.

Class Name: User	
Responsibilities: ▪ This class is responsible for storing the information of a user and adding it to the database.	Collaborations: ▪ The class User doesn't depend on or associate with any class.

Class Name: IAuthorRepository	
Responsibilities: This class is responsible for querying in the table of FavourableAuthors.	Collaborations: The class IAuthorRepository depends on the FavourableAuthor class.

Class Name: IBookAuthorRepository	
Responsibilities: This class is responsible for querying in the table of Authors.	Collaborations: The class IBookAuthorRepository depends on the BookAuthor class.

Class Name: IBookRepository	
Responsibilities: This class is responsible for querying in the table of Books.	Collaborations: The class IBookRepository depends on the Book class.

Class Name: IFavourableCategoryRepository	
Responsibilities: This class is responsible for querying in the table of FavourableCategories.	Collaborations: The class IFavourableCategoryRepository depends on the FavourableCategory class.

Class Name: INotificationRepository	
Responsibilities: This class is responsible for querying in the table of Notifications.	Collaborations: The class INotificationRepository depends on the Notification class.

Class Name: IOfferRepository	
Responsibilities: This class is responsible for querying in the table of Offers.	Collaborations: The class IOfferRepository depends on the Offer, OfferKey and User classes.

Class Name: IRequestRepository	
Responsibilities: ▪ This class is responsible for querying in the table of Requests.	Collaborations: ▪ The class IRequestRepository depends on the Request class.

Class Name: IUserRepository	
Responsibilities: ▪ This class is responsible for querying in the table of Users.	Collaborations: ▪ The class IUserRepository depends on the User class.

Class Name: SocialBookstoreApplication	
Responsibilities: ▪ This class is responsible for running the Spring application.	Collaborations: ▪ The class SocialBookstoreApplication doesn't depend on or associate with any class.